Niz računa sa dvije vrste računa

Banci treba spisak svih računa, bez obzira na vrstu. Jedino što im je zajedničko jeste da su svi **Racun**.

```
int main()
{
    ZiroRacun   z("501-001", 1000, 500);
    StedniRacun s("501-003", 2000, 2.5);

    Racun* racuni[2] = { &z, &s };

    for (int i = 0; i < 2; i++)
        racuni[i]->prikazi();
}
```

Šta nam ovo omogućava

Tip elementa je **Racun***, a u njega staje adresa **bilo koje izvedene klase**.

Petlja ne zna čime barata. Koju verziju metode **prikazi** će pozvati? Pozvaće se verzija iz klase **Racun**. To je **statičko vezivanje.**

Statičko vezivanje

ispis programa

```
501-001: 1000 EUR
501-003: 2000 EUR
```

Dodaci *stopa* i *minus* su nestali

Ni jedan ni drugi račun nije prikazao svoj specifični dio. Pozvana je verzija iz klase **Racun**, oba puta.

Statičko vezivanje

Poziv **obične** metode prevodilac razrješava po **tipu pokazivača**, a ne po tipu objekta. Tip pokazivača je **Racun***, pa se bira verzija iz osnovne klase **Racun::prikazi**. Odluka o verziji metode se donosi u vrijeme prevođenja i više se ne mijenja — to je **statičko vezivanje** (engl. *early binding*).

Međutim mi bismo htjeli da metoda bude izabrana po **tipu objekta** na koji pokazivač pokazuje, u vrijeme izvršavanja. To je **polimorfizam**, a u C++ se to postiže ključnom riječju **virtual** pri deklaraciji metode u osnovnoj klasi.

Polimorfizam

Polimorfizam je svojstvo da objekat izvedene klase izvrši metodu onako kako je ona definisana **u njegovoj klasi**, iako mu se pristupa kao objektu osnovne klase.

Na osnovu čega se određuje verzija metode

Koja će se verzija metode pozvati određuje tip **objekta** na koji pokazivač ukazuje, a **ne tip pokazivača**. Svaki objekat sam prepozna kojoj klasi pripada, iako mu se obraćamo kao objektu osnovne klase.

Kada se određuje verzija metode

U vrijeme **izvršavanja**, a ne u vrijeme prevođenja. Zato se zove i kasno vezivanje (*late binding*).

Čime se postiže polimorfizam u C++

Virtuelnim metodama. Osnovna klasa označi metodu riječju **virtual** i time kaže da izvedena klasa smije zamijeniti tu metodu svojom verzijom.

Klučna riječ *virtual*

osnovna klasa

```
class Racun {
protected:
    std::string broj;
    double stanje;
public:
    Racun(std::string broj, double stanje)
        : broj(broj), stanje(stanje) {}
    virtual void prikazi() const    // jedina izmjena
    { std::cout << broj << ": "
                << stanje << " EUR\n"; }
};
```

Šta smo promijenili u kodu?

Samo deklaraciju metode u osnovnoj klasi. Klase **ZiroRacun** i **StedniRacun** ostaju iste, i **main** ostaje isti.

Terminologija

Metoda označena sa **virtual** je **virtuelna** ili **polimorfna metoda**. Klasa koja ima bar jednu takvu metodu je **polimorfna klasa**. U izvedenoj klasi riječ **virtual** se ne mora ponavljati — metoda ostaje virtuelna.

Ispis sa virtuelnom funkcijom

ispis programa

```
501-001: 1000 EUR
  minus do 500
501-003: 2000 EUR
  stopa 2.5%
```

Pravilo

Verzija virtuelne metode razrješava se po **tipu objekta** na koji pokazivač ukazuje, u vrijeme izvršavanja. To je **dinamičko vezivanje** (engl. *late binding*).

Deklaracije moraju biti iste u osnovnoj i izvedenoj klasi

Da bi metoda izvedene klase zamijenila virtuelnu metodu osnovne, moraju se podudarati **ime metode, broj i tipovi argumenata**, **const** i **povratni tip**. Ako se bilo šta od toga razlikuje, nastaje nova metoda koja samo sakriva metodu iz osnovne klase

Dinamičko vezivanje radi samo kroz pokazivač ili referencu, dinamičko vezivanje nije rješenje za odsijecanje objekta

```
void ispisi(Racun r)        // prenos po vrijednosti
{
    std::cout << "u funkciji, velicina: "
              << sizeof(r) << "\n";
    r.prikazi(); // poziva se verzija iz klase Racun
}

int main()
{
    StedniRacun s("501-003", 1000, 2.5);
    std::cout << "u mainu, velicina:    "
              << sizeof(s) << "\n";
    s.prikazi();
    ispisi(s);
}
```

ispis programa

```
u mainu, velicina:   56
501-003: 1000 EUR
  stopa 2.5%
u funkciji, velicina: 48
501-003: 1000 EUR
```

Napomena

Dinamičko vezivanje radi samo kroz **pokazivač** ili **referencu**. Objekat prenesen po **vrijednosti** je **odsječen** još prije poziva, pa tu ni **virtual** neće učiniti da se pozove verzija metode iz izvedene klase. Iako je metoda deklarisana kao virtuelna u klasi Racun, u funkciji *ispisi* pozvaće se metoda iz osnovne klase, ne iz izvedene, jer je došlo do odsijecanja objekta.

Dok se dodjeljuju pokazivači, nema odsijecanja. Kada se dodjeljuju i kopiraju objekti, imamo odsijecanje.

Ako se potpisi metoda ne slažu potpuno

Osnovna klasa je nepromijenjena, sa **virtual void prikazi() const**. U izvedenoj klasi **izostavili** smo jednu riječ: **const**

```
class StedniRacun : public Racun {
    double stopa;
public:
    StedniRacun(std::string broj, double stanje,
                double stopa)
        : Racun(broj, stanje), stopa(stopa) {}
    void prikazi()              // nedostaje const
    { Racun::prikazi();
      std::cout << "  stopa " << stopa << "%\n"; }
};

int main()
{
    StedniRacun s("501-003", 2000, 2.5);
    Racun* p = &s;
    p->prikazi();
}
```

ispis programa

```
501-003: 2000 EUR
```

void prikazi() iz izvedene klase i **void prikazi() const** iz osnovne klase su dvije različite metode. Izvedena klasa nije zamijenila virtuelnu metodu nego je napravila **novu**, pa poziv preko **Racun*** ide na verziju osnovne klase iako je verzija iz osnovne klase virtuelna.

Taj problem rješavamo ključnom riječju *override* u izvedenoj klasi

Isti kod sa prethodnog slajda, uz jednu dopisanu riječ: **override**. Program se više ne prevodi.

```
class StedniRacun : public Racun {
    double stopa;
public:
    StedniRacun(std::string broj, double stanje,
                double stopa)
        : Racun(broj, stanje), stopa(stopa) {}
    void prikazi() override   // tvrdnja prevodiocu
    { Racun::prikazi();
      std::cout << "  stopa " << stopa << "%\n"; }
};
```

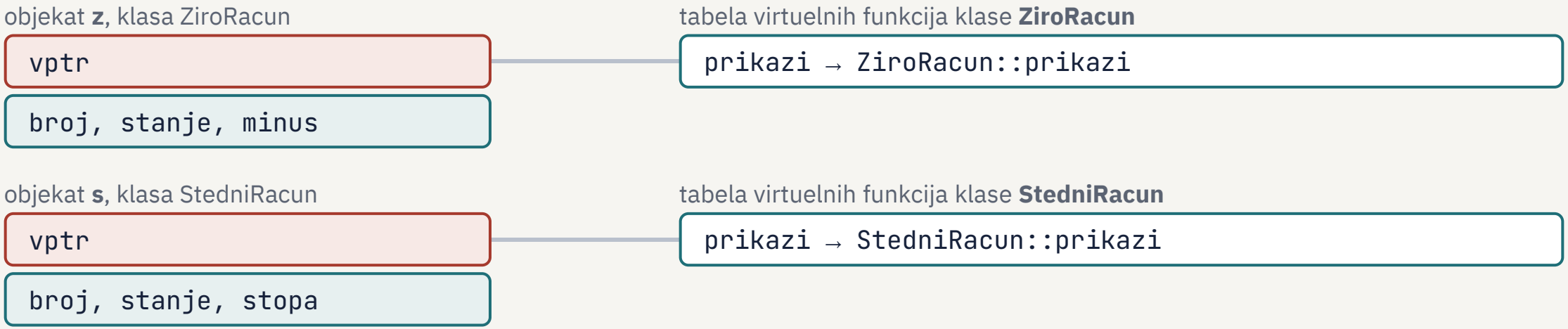
```
error: 'void StedniRacun::prikazi()' marked
       'override', but does not override
```

ispravka: dodamo const i potpisi se sada poklapaju

```
void prikazi() const override   // slazu se potpisi
```

Override kod metode izvedene klase znači: ova metoda zamjenjuje neku virtuelnu metodu osnovne klase. Prevodilac onda provjerava, i ako se potpisi ne poklapaju u svemu — **imenu**, **argumentima**, **const** i **povratnom tipu** — prijavljuje grešku.
Preporuka: *override* pisati u izvedenoj klasi, uz svaku redefinisanu virtuelnu metodu.

Kako se dinamičko vezivanje izvodi



Šta se dešava pri pozivu *p->prikazi()*

Poziv **p->prikazi()** ne skoči na unaprijed poznatu adresu već se iz objekta pročita ***vptr* – pokazivač na tabelu virtuelnih funkcija**, iz tabele se pročita adresa metode, pa se skoči na nju. Pokazivač **vptr** setuje konstruktor, pa objekat od svog nastanka nosi tabelu svoje klase.

Dodatno memorijsko i komputaciono opterećenje

Kod dinamičkog vezivanja imamo **jedan pokazivač više (vptr)** po objektu i **jedno čitanje više** pri svakom pozivu. Zato **virtual** nije podrazumijevano u C++ nego se traži izričito.

Konstruktor ne može biti virtuelan

Razlog: u trenutku njegovog poziva *vptr* još nije upisan pa se ne bi mogla razriješiti verzija virtuelnog konstruktora – u trenutku poziva konstruktora još uvijek ne postoji objekat pa ne postoji ni vptr.

Koliko skriveni pokazivač *vptr* zauzima memorije

```
class BezVirtuelne {
    std::string broj;
    double stanje;
public:
    void prikazi() const;
};

class SaVirtuelnom {
    std::string broj;
    double stanje;
public:
    virtual void prikazi() const;
    virtual ~SaVirtuelnom();
};
```

```
int main()
{
    std::cout << "bez virtuelne metode: "
              << sizeof(BezVirtuelne)
              << " bajtova\n";
    std::cout << "sa virtuelnom:        "
              << sizeof(SaVirtuelnom)
              << " bajtova\n";
}
```

ispis programa

```
bez virtuelne metode: 40 bajtova
sa virtuelnom:        48 bajtova
```

Obje klase imaju iste atribute. Razlika od **8 bajtova** je **vptr**. Dodaje se jednom po objektu, bez obzira na broj virtuelnih metoda.

Klasa čije objekte ne možemo kreirati

Imamo novi zahtjev od banke: Račun uvijek jeste neke vrste (npr. štedni ili žiro). Objekat koji je samo **Racun** ne postoji ni u banci ni u programu.

Čisto virtuelna metoda

Virtuelna metoda **bez tijela**, umjesto kojeg se piše **= 0**. Osnovna klasa time kaže da metoda postoji i da je svaka izvedena klasa **mora** zasebno definisati.

Apstraktna klasa

Klasa sa bar jednom čisto virtuelnom metodom je apstraktna klasa. Njen objekat se **ne može napraviti**, ali pokazivač i referenca na nju postoje i pokazuju na konkretne objekte izvedenih klasa.

Šta sve može da ima apstraktna klasa

Atribute, obične metode, obične virtuelne metode i konstruktor. Konstruktor se izvršava kao dio konstrukcije objekta izvedene klase.

Nasljeđivanje obaveze da se definiše čisto virtuelna funkcija

Čisto virtuelna metoda se nasljeduje kao čisto virtuelna. Ako je izvedena klasa ne definiše, i ona je apstraktna, pa ni njeni objekti ne postoje (ne možemo ih kreirati).

Apstraktna klasa Racun i dvije konkretne klase

apstraktna osnovna klasa

```
class Racun {            // apstraktna klasa
protected:
    std::string broj;
    double stanje;
public:
    Racun(std::string broj, double stanje)
        : broj(broj), stanje(stanje) {}
    virtual ~Racun() {}
    virtual double naknada() const = 0;
    void prikazi() const
    { std::cout << broj << ": " << stanje
                << " EUR, naknada "
                << naknada() << "\n"; }
};
```

dvije konkretne klase

```
class ZiroRacun : public Racun {
    double minus;
public:
    ZiroRacun(std::string broj, double stanje,
              double minus)
        : Racun(broj, stanje), minus(minus) {}
    double naknada() const override
    { return 1.5; }
};

class StedniRacun : public Racun {
    double stopa;
public:
    StedniRacun(std::string broj, double stanje,
                double stopa)
        : Racun(broj, stanje), stopa(stopa) {}
    double naknada() const override
    { return 0.0; }
};
```

Apstraktna klasa Racun i dvije konkretne klase

```
int main()
{
    ZiroRacun   z("501-001", 1000, 500);
    StedniRacun s("501-003", 2000, 2.5);
    Racun* racuni[2] = { &z, &s };

    double ukupno = 0;
    for (int i = 0; i < 2; i++)
    {
        racuni[i]->prikazi();
        ukupno += racuni[i]->naknada();
    }
    std::cout << "ukupna naknada: "
              << ukupno << " EUR\n";
}
```

ispis programa

```
501-001: 1000 EUR, naknada 1.5
501-003: 2000 EUR, naknada 0
ukupna naknada: 1.5 EUR
```

drugi program: objekat apstraktne klase nije moguće napraviti

```
int main()
{
    Racun r("501-000", 100);
}
```

```
error: cannot declare variable 'r' to be of
       abstract type 'Racun'
note:   because the following virtual functions
        are pure within 'Racun':
note:     'virtual double Racun::naknada() const'
```

Kada pokušamo da napravimo objekat apstraktne klase, prevodilac prijavljuje grešku i nabraja koje su metode ostale **čisto virtuelne**. Ista poruka pojaviće se i za izvedenu klasu koja ne definiše makar jednu **čisto virtuelnu metodu**, jer je i ona tada apstraktna.

Apstraktna klasa je *ugovor* koji se izvedene klase obavezuju da će ispuniti.

Izvedena klasa koja drži resurs

Statički brojač **brojStednihRacuna** raste u konstruktoru i pada u destruktoru, pa na kraju programa mora biti nula.

osnovna klasa

```
class Racun {
protected:
    std::string broj;
    double stanje;
public:
    Racun(std::string broj, double stanje)
        : broj(broj), stanje(stanje) {}
    ~Racun()              // nije virtuelan
    { std::cout << "Racun: destruktor\n"; }
    virtual void prikazi() const
    { std::cout << broj << ": "
                << stanje << " EUR\n"; }
};
```

izvedena klasa sa nizom na heapu

```
class StedniRacun : public Racun {
    double stopa;
    double* istorija;      // 12 mjesecnih stanja
public:
    static int brojStednihRacuna;
    StedniRacun(std::string broj, double stanje,
                double stopa)
        : Racun(broj, stanje), stopa(stopa),
          istorija(new double[12])
    { brojStednihRacuna++; }
    ~StedniRacun()
    { delete[] istorija;
      brojStednihRacuna--;
      std::cout << "StedniRacun: destruktor\n"; }
    void prikazi() const override
    { Racun::prikazi();
      std::cout << "  stopa " << stopa << "%\n"; }
};
int StedniRacun::brojStednihRacuna = 0;
```

Destruktor izvedene klase nije pozvan zbog statičkog vezivanja

```
int main()
{
    Racun* p =
        new StedniRacun("501-003", 2000, 2.5);
    p->prikazi();
    delete p;
    std::cout << "stednih racuna: "
              << StedniRacun::brojStednihRacuna
              << "\n";
}
```

ispis programa

```
501-003: 2000 EUR
  stopa 2.5%
Racun: destruktor
stednih racuna: 1
```

```
warning: deleting object of polymorphic
  class type 'Racun' which has non-virtual
  destructor might cause undefined behavior
  [-Wdelete-non-virtual-dtor]
```

Operator **delete** radi preko pokazivača tipa **Racun***, pa po **statičkom** vezivanju poziva samo **~Racun**. Destruktor izvedene klase nikada nije pozvan, pa niz na heap-u ostaje zauzet, a brojač ostaje setovan na 1 iako više nema nijednog štednog računa.

Dodajemo *virtual* u deklaraciji destruktora

u klasi Racun

```
    virtual ~Racun()      // jedina izmjena
    { std::cout << "Racun: destruktor\n"; }
```

ispis programa

```
501-003: 2000 EUR
  stopa 2.5%
StedniRacun: destruktor
Racun: destruktor
stednih racuna: 0
```

Kad je destruktor virtuelan, **delete** ga bira po tipu **objekta**, a ne po tipu pokazivača. Pokreće se destruktor klase **StedniRacun**, koji oslobađava niz, pa zatim automatski i destruktor osnovne klase — obrnuti redoslijed. Brojač je sada 0 i prevodilac ne daje upozorenje.

Preporuka

Ako klasa ima bar jednu virtuelnu metodu, njen destruktor treba da bude virtuelan.

Razlog:

Virtuelna metoda znači **da se klasa koristi preko pokazivača** na osnovnu klasu. A ako se tako koristi, pozvaće se i **delete** preko istog pokazivača.

Nema dodatnog memorijskog opterećenja

Ako klasa već ima virtuelnih metoda — pokazivač na tabelu već postoji.

Konstruktor ne može biti virtuelan

Virtuelni poziv se razrješava preko tabele, a pokazivač na tabelu upisuje upravo konstruktor. U trenutku njegovog poziva, nema objekta pa nema ni tabele.

Veza sa pravilom tri

Klasa koja ima destruktor, konstruktor kopije i operator dodjele dobija ovdje četvrtu stavku: ako je ona osnovna klasa, taj destruktor mora biti **virtuelan**.

Operator typeid

Vraća **const type_info&**, objekat koji opisuje **stvarni tip izraza**. Za polimorfne klase to je tip objekta, a ne tip pokazivača.

```
#include <typeinfo>

int main()
{
    ZiroRacun   z("501-001", 1000, 500);
    StedniRacun s("501-003", 2000, 2.5);
    Racun* racuni[2] = { &z, &s };

    for (int i = 0; i < 2; i++)
    {
        std::cout << "name(): "
                  << typeid(*racuni[i]).name() << "\n";

        if (typeid(*racuni[i]) == typeid(StedniRacun))
            std::cout << "  ovo je stedni racun\n";
    }
}
```

ispis programa

```
name(): 9ZiroRacun
name(): 11StedniRacun
  ovo je stedni racun
```

Rezultati se mogu porediti operatorima **==** i **!=** i tako je najsigurnije porediti ih. Metoda **name()** vraća ime u obliku koji zavisi od prevodioca pa nije sigurno porediti imena tipova — u ovom slučaju su brojevi ispred imena dužina imena.

Operator dynamic_cast

Preko **Racun*** vidimo samo ono što nudi **Racun**. Metoda **pripisiKamatu** postoji samo na štednom računu, a u nizu su obje vrste racuna.

```
int main()
{
    ZiroRacun   z("501-001", 1000, 500);
    StedniRacun s("501-003", 2000, 2.5);
    Racun* racuni[2] = { &z, &s };

    for (int i = 0; i < 2; i++)
    {
        StedniRacun* sr =
            dynamic_cast<StedniRacun*>(racuni[i]);
        if (sr != nullptr)
            sr->pripisiKamatu();
        racuni[i]->prikazi();
    }
}
```

dodato u izvedenu klasu StedniRacun

```
void pripisiKamatu()   // u StedniRacun
{ stanje += stanje * stopa / 100; }
```

ispis programa

```
501-001: 1000 EUR
  minus do 500
501-003: 2050 EUR
  stopa 2.5%
```

dynamic_cast<StedniRacun*>(p) u vrijeme izvršavanja provjerava da li objekat na koji p ukazuje jeste štedni račun. Ako jeste, dobijamo pokazivač na taj isti objekat, ali tipa StedniRacun*. Ako nije, dobijamo nullptr.

Česta potreba za ovim obično znači da je neka metoda trebalo da bude virtuelna.